Image Segmentation

Module Code: 7PAM2015-0509-2024

Student Name: Revathi Nanubala

Student ID: 22032334

Word Count: 952

Google Colab Link: https://colab.research.google.com/drive/19Rw8_uaC5D9O3Lc-eMKZgs4idRBSiIR0?usp=sharing

Introduction and Literature Review

Semantic segmentation associates each pixel to a class that is essential for the object boundary description in various fields including medical imaging, autonomous driving, and scene interpretation (Long et al., 2015). Unlike classification, it offers spatial context for the learning of the object structure.

Deep learning has brought a revolution in segmentation, especially with CNNs. Dense predictions gave birth to Fully Convolutional Networks (Long et al., 2015), and U-Net's encoder-decoder with skip connections proved invaluable in several domains (Ronneberger et al., 2015).

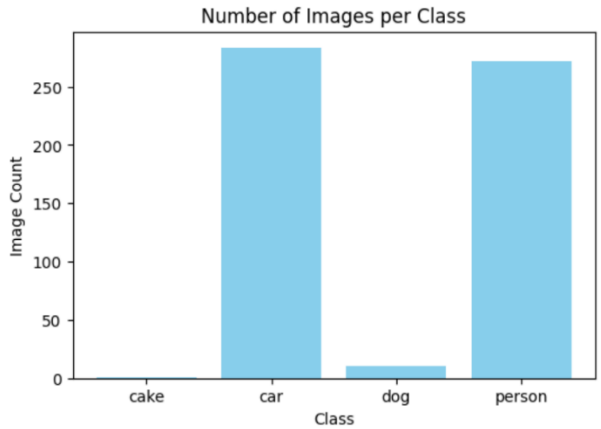
This project applies U-Net for segmenting classes of "cake", "car", "dog", and "person" from COCO 2017 dataset (Lin et al., 2014), giving challenges like scale variations and occlusion. The annotations in COCO format were used to create the 128×128 masks, which were then improved with augmentation techniques such as flipping and rotation for better generalization (Shorten & Khoshgoftaar, 2019).

Data Set and Exploratory Data Analysis

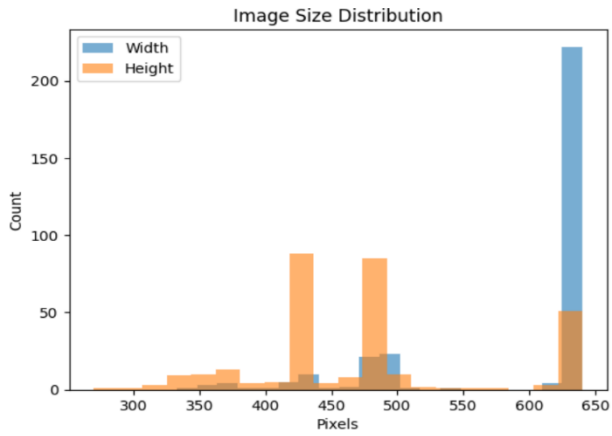
It uses a curated version of COCO dataset 2017 with 630 labeled images, which have been divided into train (300), val (300), and test (30). The classes were chosen such that each class has a different visual appearance and yet remains simple enough for segmentation: cake, car, dog, and person. Annotations are in the COCO standard (Lin et al., 2014) format, including pixel-level segmentation through polygons.

Class distribution in the training set is imbalanced:

- **Cake**: 1 image
- **Car**: 283 images
- **Dog**: 10 images
- **Person**: 272 images



The different image resolutions exhibit a huge range of heights and widths, which were constants for creating generalization problems for fixed-size models.



Analysis of annotation frequency showed "person" to have high representation, which may influence learning.

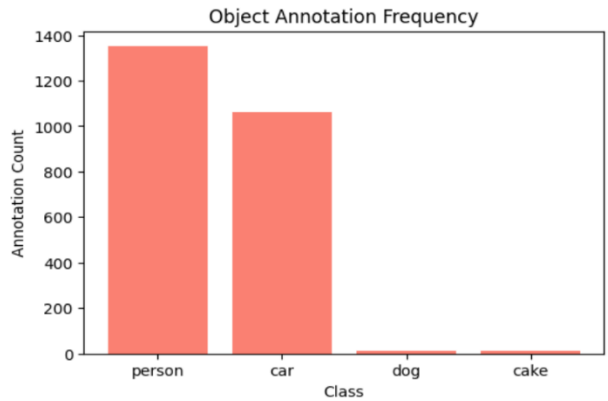
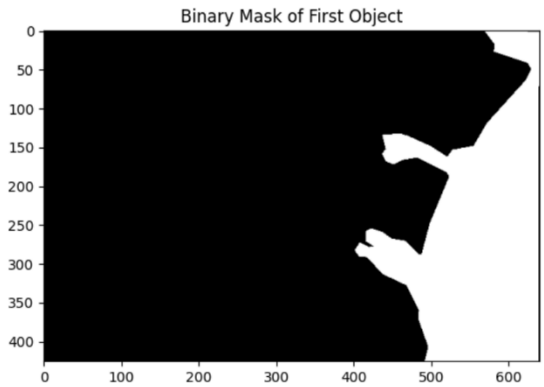
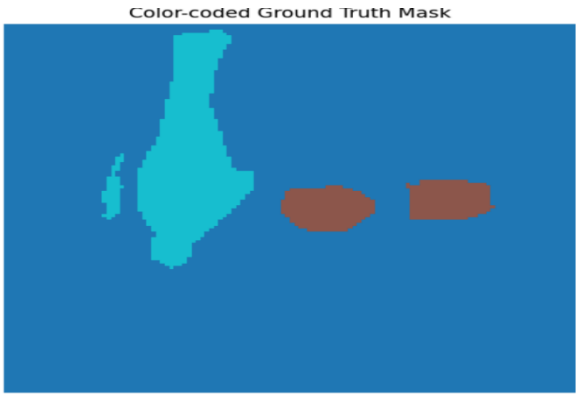


Figure shows illustrative samples of objects from the targeted classes within the training dataset. The image is overlaid with bounding boxes and segmentation masks showing the ground truth annotations. These segmentation masks define pixel-wise labels for every instance of object that can be used for the supervised training of segmentation models.



All images and masks were resized to 128×128. Data augmentations like flips and 90° rotations were applied to add robustness (Shorten & Khoshgoftaar, 2019). Finally, as below, the model is validated with a color-coded ground-truth mask.



The complete collection comprises 276 images featuring the target classes. Sample masks are supposed to be binary Numpy arrays sitting in a numpy structure that indicated the object's regions. The diversity plays an important role in making a robust semantic segmentation model despite class imbalance.

Methodology and Approach to Implementation

This project performs semantic segmentation on four target classes (cake, car, dog, person) using a subset of the COCO 2017 dataset. Data preparation (labeling) is done, U-Net is trained (modified), and performance is tested.

Data Processing

COCO annotation is parsed in order to include images that have at least one target class. Then multi-class segmentation masks are created for them, using a 0 value for background and assigning an integer value (1–4) to each class. Next, images and masks will be resized to 128×128. The data will undergo augmentation (shorten & khoshgoftaar, 2019), namely, random flips and rotation 90°,, 180°, 270°.

Model Architecture

An improved U-Net model is utilized here which is composed of 3 encoding blocks and corresponding decoding

blocks with skip connections. Following every convolution, a batch normalization layer and ReLU activation are executed. The last layer outputs softmax probabilities for five classes (background + four targets).

Loss and Metrics

To warrant class imbalance, a custom weighted sparse categorical cross-entropy loss is used with pre-defined class weights. In addition to that, a custom mean IoU (mIoU) metric is available for evaluation and computed by averaging IoUs across all classes (Everingham et al., 2010).

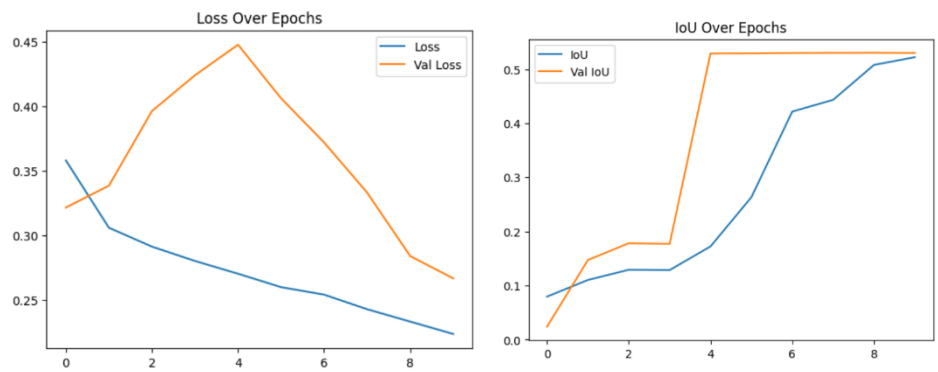
Training Procedure

Dataset division is 80-20 for training to validation. The model trains for 10 epochs at a batch size of 8 and is run on the Adam optimizer. There will be no data augmentation for validation.

Results and Evaluation

Training and Validation Performance

At first, the training and the validation loss diverge slowly. Eventually, both losses decrease and improve generalization due to early overfitting. Meanwhile, the mIoU improves slowly while quickly stabilizing in validation mIoU, demonstrating segmentation performance stability and effectiveness.



Quantitative Metrics

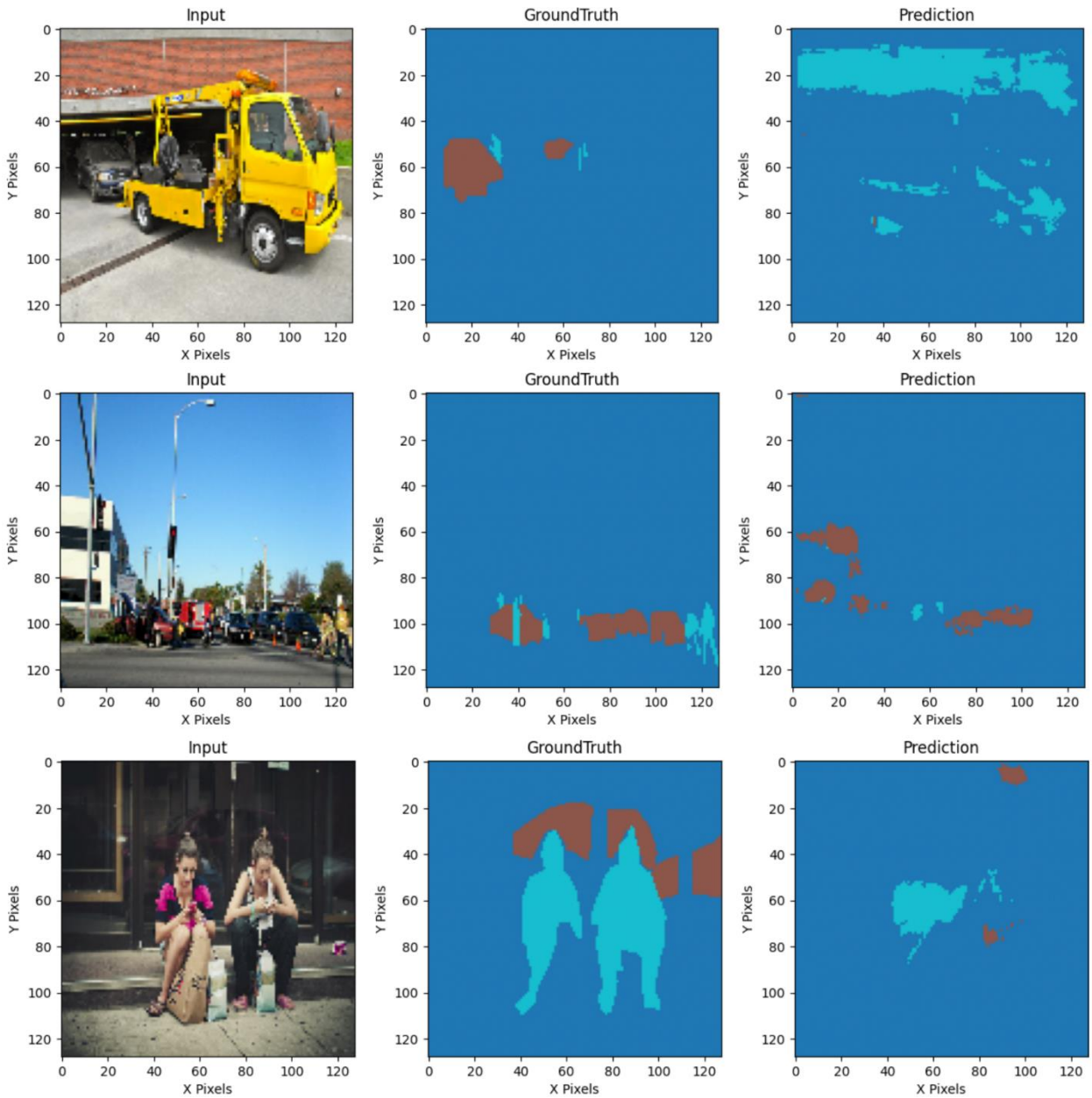
The per-class mean IoU scores obtained from the validation set are summarized in the table below.

Class	**Mean IoU Score**
Background	0.8367
Cake	1.0000
Car	0.0344
Dog	0.9661
Person	0.0639

The model is best in the Cake class and has relatively less accuracy in the Car class, probably due to different characteristics of frequency or complexity of the objects.

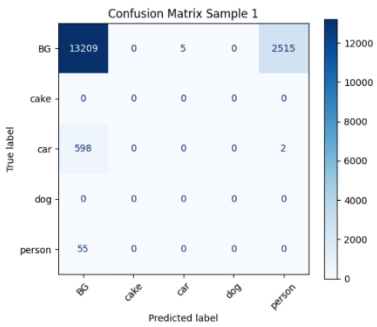
Qualitative Visualizations

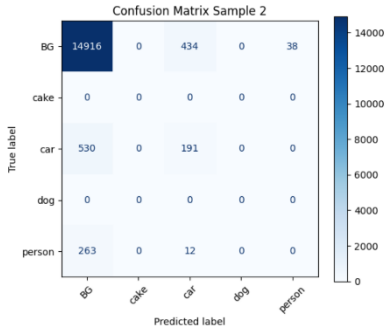
Sample predictions done on validation images visually confirm model capability of drawing object boundaries and segmenting multiple classes.

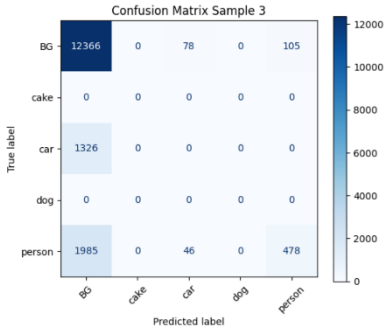


Confusion Matrix Analysis

The confusion matrices built for the chosen samples illustrate the pixel-wise classification accuracy and demonstrate some confusions between Car, Person and Background, which may be the result of texture similarities or overlapping features.







Summary and Conclusion

The advanced U-Net model exhibited high performance in semantic segmentation as indicated by increasing mIoU and decreasing loss with respect to the epochs. Data augmentation imbued the model with improved robustness against scale and orientation. Per-class IoU and confusion matrices showed good multi-class segmentation, but fine boundary accuracy is still a challenge. The project was able to segment four COCO classes—cake, car, dog, and person—which were validated by both metrics and visual outputs.

Future Work

With encouraging results, future work will involve availing the dataset with an eye to increase the range of classes, investigate advanced architectures (e.g., DeepLabv3+, SegFormer), and hyperparameter tuning. The results can be improved with post-processing (e.g., CRFs), and it would be worthwhile to optimize models for real-time applications through methods such as pruning and lightweight models, perhaps facilitating their wider deployment in autonomous driving and surveillance.

References

- Everingham, M., Van Gool, L., Williams, C.K.I., Winn, J. and Zisserman, A., 2010. The Pascal Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision*, 88(2), pp.303-338. Available at: https://link.springer.com/article/10.1007/s11263-009-0275-4
- Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P. and Zitnick, C.L., 2014. Microsoft COCO: Common Objects in Context. *In European Conference on Computer Vision (ECCV)*, pp. 740-755. Available at: https://doi.org/10.1007/978-3-319-10602-1_48
- Long, J., Shelhamer, E. and Darrell, T., 2015. Fully Convolutional Networks for Semantic Segmentation. *In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3431-3440. Available at: https://doi.org/10.1109/CVPR.2015.7298965
- Ronneberger, O., Fischer, P. and Brox, T., 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. Available at: https://arxiv.org/abs/1505.04597
- Shorten, C. and Khoshgoftaar, T.M., 2019. A survey on Image Data Augmentation for Deep Learning. *Journal of Big Data*, 6(1), p.60. Available at: https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0197-0

Appendix:

```
# —— 1. Imports & Setup ——
import os
import random
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from pycocotools.coco import COCO
import skimage.io as io
import tensorflow as tf
from tensorflow.keras import layers, models, backend as K
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
%matplotlib inline
# —— 2. Mount Drive & Unzip Dataset ——
from google.colab import drive
drive.mount("/content/drive")

!unzip -q "/content/drive/MyDrive/RM_Segmentation_Assignment_dataset.zip" \
    -d "/content/drive/MyDrive/coco2017/"
# —— 3. Load COCO Annotations ——
train_data_path = "/content/drive/MyDrive/coco2017/train-300"
train_annotation_file = os.path.join(train_data_path, "labels.json")
coco = COCO(train_annotation_file)

# —— 4. Explore Categories & Supercategories ——
cat_ids_all = coco.getCatIds()
cats = coco.loadCats(cat_ids_all)
print("COCO categories ({}):".format(len(cats)),
    ", ".join([c["name"] for c in cats]))
supercats = sorted({c["supercategory"] for c in cats})
print("Supercategories ({}):".format(len(supercats)), ", ".join(supercats))

# Utility: category name lookup
def get_category_name(cat_id):
  for c in cats:
    if c["id"] == cat_id:
      return c["name"]
  return None

# Sample category name lookups
for test_id in [10, 0, 15, 18, 25, 41, 58, 62, 16]:
  print(f"Category name for ID {test_id}: {get_category_name(test_id)}")
```

```python
# —— 5. Select Target Classes & Image Lists ——
target_classes = ["cake", "car", "dog", "person"]
target_cat_ids = coco.getCatIds(catNms=target_classes)

# Per-class image counts and IDs
for cls in target_classes:
    ids = coco.getImgIds(catIds=coco.getCatIds(catNms=[cls]))
    print(f"{cls}: {len(ids)} images, sample IDs = {ids[:5]}")

# Load all unique images containing at least one target object
all_images = []
for cls in target_classes:
    for img_info in coco.loadImgs(coco.getImgIds(catIds=coco.getCatIds(catNms=[cls]))):
        all_images.append(img_info)
# Remove duplicates
unique_images = {img["id"]: img for img in all_images}.values()
unique_images = list(unique_images)
print("Unique images containing target classes:", len(unique_images))
random.seed(0)
random.shuffle(unique_images)
# —— 6. Display One Sample Image + Annotations ——
sample = unique_images[0]
print("Sample image info:", sample)
img = io.imread(os.path.join(train_data_path, "data", sample["file_name"]))
plt.figure(figsize=(8,6))
plt.axis("off")
plt.imshow(img)
plt.title("Sample Image")

# Load & show annotations
ann_ids = coco.getAnnIds(imgIds=sample["id"], catIds=target_cat_ids, iscrowd=None)
anns = coco.loadAnns(ann_ids)
print("Annotations for this image:")
for ann in anns:
    print(f"- cat_id={ann['category_id']} "
        f"({get_category_name(ann['category_id'])}), bbox={ann['bbox']}, "
        f"{'segmentation N/A' if 'segmentation' not in ann else 'segmentation present'}")

# Image with Bounding Boxes
plt.figure(figsize=(8,6))
plt.imshow(img)
plt.axis("off")
coco.showAnns(anns, draw_bbox=True)
plt.title("Image with Bounding Boxes")
```

```
plt.show()

# Segmentation Only
plt.figure(figsize=(8,6))
plt.imshow(img)
plt.axis("off")
coco.showAnns(anns, draw_bbox=False)
plt.title("Segmentation Only")
plt.show()

# Explore annToMask output
mask_example = coco.annToMask(anns[0])
print("Mask example: type", type(mask_example),
    "shape", mask_example.shape,
    "min/max", mask_example.min(), "/", mask_example.max())
plt.imshow(mask_example, cmap="gray")
plt.title("Binary Mask of First Object")
plt.show()

# —— 7. Dataset EDA ——
# Image count per class
image_counts = {cls: len(coco.getImgIds(catIds=coco.getCatIds(catNms=[cls])))
            for cls in target_classes}
plt.figure(figsize=(6,4))
plt.bar(image_counts.keys(), image_counts.values(), color="skyblue")
plt.title("Number of Images per Class")
plt.xlabel("Class")
plt.ylabel("Image Count")
plt.show()

# Image size distribution
sizes = np.array([(img["width"], img["height"]) for img in unique_images])
plt.hist(sizes[:, 0], bins=20, alpha=0.6, label="Width")
plt.hist(sizes[:, 1], bins=20, alpha=0.6, label="Height")
plt.title("Image Size Distribution")
plt.xlabel("Pixels")
plt.ylabel("Count")
plt.legend()
plt.show()

# Object annotation frequency
ann_ids_all = coco.getAnnIds(imgIds=[img["id"] for img in unique_images],
                        catIds=target_cat_ids)
anns_all = coco.loadAnns(ann_ids_all)
```

```
freq = Counter([get_category_name(a["category_id"]) for a in anns_all])
plt.figure(figsize=(6,4))
plt.bar(freq.keys(), freq.values(), color="salmon")
plt.title("Object Annotation Frequency")
plt.xlabel("Class")
plt.ylabel("Annotation Count")
plt.show()
# —— 8. Mask Creation ——
def create_mask(img_info):
    mask = np.zeros((img_info["height"], img_info["width"]), dtype=np.uint8)
    for ann in coco.loadAnns(coco.getAnnIds(imgIds=[img_info["id"]],
                                            catIds=target_cat_ids)):
        idx = target_cat_ids.index(ann["category_id"]) + 1
        mask = np.maximum(mask, coco.annToMask(ann) * idx)
    return mask
# —— 9. Data Augmentation ——
def augment_image_and_mask(image, mask):
    # Random horizontal flip
    if random.random() > 0.5:
        image = tf.image.flip_left_right(image)
        mask = tf.image.flip_left_right(mask)
    # Random vertical flip
    if random.random() > 0.5:
        image = tf.image.flip_up_down(image)
        mask = tf.image.flip_up_down(mask)
    # Random rotation by k*90 degrees
    k = random.choice([0, 1, 2, 3])
    image = tf.image.rot90(image, k=k)
    mask = tf.image.rot90(mask, k=k)
    return image, mask
# —— 10. Preprocessing with Augmentation Option ——
def preprocess_data(img_list, size=(128,128), augment=False):
    imgs, masks = [], []
    for imf in img_list:
        try:
            img = io.imread(os.path.join(train_data_path, "data", imf["file_name"]))
            mask = create_mask(imf)
            img = tf.image.resize(img, size) / 255.0
            mask = tf.image.resize(tf.expand_dims(mask, -1), size, method="nearest")
            if augment:
                img, mask = augment_image_and_mask(img, mask)
            imgs.append(img.numpy())
            masks.append(mask.numpy().squeeze())
        except Exception as e:
```

```
        print("Skipping", imf["file_name"], "due to", str(e))
    return np.array(imgs), np.array(masks)

train_imgs, val_imgs = train_test_split(unique_images,
                                        test_size=0.2,
                                        random_state=42)
X_train, y_train = preprocess_data(train_imgs, augment=True)
X_val, y_val = preprocess_data(val_imgs, augment=False)

print("Shapes:", X_train.shape, y_train.shape)
# —— 11. Visualize Color-Mapped Mask ——
def colorize_mask(mask):
    cmap = plt.get_cmap("tab10")
    out = cmap(mask / (mask.max() or 1))
    return out

plt.figure(figsize=(6,6))
plt.imshow(colorize_mask(y_train[0]))
plt.title("Color-coded Ground Truth Mask")
plt.axis("off")
plt.show()
# —— 12. Define IoU Metric for Evaluation and Weighted Loss Function ——
def iou_coef(y_true, y_pred, num_classes=len(target_classes)+1, smooth=1e-6):
    y_true = tf.one_hot(tf.cast(tf.squeeze(y_true), tf.int32), num_classes)
    y_pred = tf.one_hot(tf.argmax(y_pred, axis=-1), num_classes)
    intersection = tf.reduce_sum(y_true * y_pred, axis=[0,1,2])
    union = tf.reduce_sum(y_true + y_pred, axis=[0,1,2]) - intersection
    iou = (intersection + smooth) / (union + smooth)
    return tf.reduce_mean(iou)

class_weights = tf.constant([0.1, 1.0, 1.0, 1.0, 1.0])

def weighted_sparse_cce(y_true, y_pred):
    y_true = tf.cast(y_true, tf.int32)
    y_true_one_hot = tf.one_hot(y_true, depth=5)
    weights = tf.reduce_sum(class_weights * y_true_one_hot, axis=-1)
    loss = tf.keras.losses.sparse_categorical_crossentropy(y_true, y_pred)
    return loss * weights
# —— 13. U-Net Model ——
def unet_model(input_size=(128,128,3), num_classes=len(target_classes)+1):
    inputs = layers.Input(input_size)
    c1 = layers.Conv2D(16, 3, activation="relu", padding="same")(inputs)
    c1 = layers.BatchNormalization()(c1)
    c1 = layers.Conv2D(16, 3, activation="relu", padding="same")(c1)
```

```
    c1 = layers.BatchNormalization()(c1)
    p1 = layers.MaxPooling2D()(c1)

    c2 = layers.Conv2D(32, 3, activation="relu", padding="same")(p1)
    c2 = layers.BatchNormalization()(c2)
    c2 = layers.Conv2D(32, 3, activation="relu", padding="same")(c2)
    c2 = layers.BatchNormalization()(c2)
    p2 = layers.MaxPooling2D()(c2)

    c3 = layers.Conv2D(64, 3, activation="relu", padding="same")(p2)
    c3 = layers.BatchNormalization()(c3)
    c3 = layers.Conv2D(64, 3, activation="relu", padding="same")(c3)
    c3 = layers.BatchNormalization()(c3)

    u1 = layers.UpSampling2D()(c3)
    u1 = layers.concatenate([u1, c2])
    c4 = layers.Conv2D(32, 3, activation="relu", padding="same")(u1)
    c4 = layers.BatchNormalization()(c4)

    u2 = layers.UpSampling2D()(c4)
    u2 = layers.concatenate([u2, c1])
    c5 = layers.Conv2D(16, 3, activation="relu", padding="same")(u2)
    c5 = layers.BatchNormalization()(c5)

    outputs = layers.Conv2D(num_classes, 1, activation="softmax")(c5)
    return models.Model(inputs, outputs)

model = unet_model()
model.compile(optimizer="adam",
              loss=weighted_sparse_cce,
              metrics=["accuracy", iou_coef])

# —— 14. Train ——
history = model.fit(X_train, y_train,
                    validation_data=(X_val, y_val),
                    epochs=10, batch_size=8)
# —— 15. Visualize Metrics ——
plt.plot(history.history['loss'], label='Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss Over Epochs')
plt.legend()
plt.show()

plt.plot(history.history['iou_coef'], label='IoU')
```

```
plt.plot(history.history['val_iou_coef'], label='Val IoU')
plt.title('IoU Over Epochs')
plt.legend()
plt.show()
 # —— 16. Predictions & Visual Comparison ——
for i in range(3):
    img = X_val[i]
    gt = y_val[i]
    pred = np.argmax(model.predict(tf.expand_dims(img, 0))[0], axis=-1)
    plt.figure(figsize=(12,4))
    for j, (d, ttl) in enumerate(zip([img, gt, pred], ["Input", "GroundTruth", "Prediction"])):
        plt.subplot(1, 3, j+1)
        plt.imshow(d if j == 0 else colorize_mask(d))
        plt.title(ttl)
        plt.xlabel("X Pixels"); plt.ylabel("Y Pixels")
        plt.axis("on")
    plt.tight_layout()
    plt.show()
# —— 17. Per-Class IoU ——
def per_class_iou(y_true, y_pred, num_classes=len(target_classes)+1, smooth=1e-6):
    ious = []
    for cls in range(num_classes):
        true_cls = (y_true == cls).astype(np.int32)
        pred_cls = (y_pred == cls).astype(np.int32)
        intersection = np.sum(true_cls * pred_cls)
        union = np.sum(true_cls) + np.sum(pred_cls) - intersection
        iou = (intersection + smooth) / (union + smooth)
        ious.append(iou)
    return ious

label_names = ["BG"] + target_classes
ious_all = []
for i in range(len(X_val)):
    true_mask = y_val[i]
    pred_mask = np.argmax(model.predict(tf.expand_dims(X_val[i], 0))[0], axis=-1)
    ious = per_class_iou(true_mask, pred_mask)
    ious_all.append(ious)

mean_ious = np.mean(ious_all, axis=0)
for cls_id, iou_score in enumerate(mean_ious):
    print(f"{label_names[cls_id]}: {iou_score:.4f}")
# —— 18. Confusion Matrices ——
for i in range(3):
    true = y_val[i].flatten()
```

```
pred = np.argmax(model.predict(tf.expand_dims(X_val[i], 0))[0], axis=-1).flatten()
cm = confusion_matrix(true, pred, labels=labels)
disp = ConfusionMatrixDisplay(cm, display_labels=label_names)
plt.figure(figsize=(6,5))
disp.plot(cmap="Blues", xticks_rotation=45, ax=plt.gca())
plt.title(f"Confusion Matrix Sample {i+1}")
plt.show()
```